PAGINA WEB

React JS
Manejo de Datos

- Condicional if
- Operador condicional ?
- Operador &&
- Método Map()
- key
- Método Filter()

Profesora:
Elizabeth Gisela Oviedo Luna

FLUJO DE DATOS

En React el **flujo de datos es unidireccional,** los componentes superiores propagan los datos a los que están en un orden inferior, estos trabajarán con esos datos y cuando cambian su estado envían los eventos hacia los de orden superior para actualizarse.

En React, controlas la lógica de flujo de datos con JavaScript.

Condicional if

Se puede utilizar la expresion if para condicionales de JSX.

Puede guardar condicionalmente algo de JSX en una variable y luego incluirlo dentro de otro JSX usando llaves.

```
function Empleado(empleado) {
  let estado="Inactivo";
  if (empleado.activo)
  {  estado="Activo";   }
  return (<li>{empleado.nombre} - {empleado.cargo} - {estado}</li> )
}

function Listado() {
  return (
    <section>
    <h3>Listado de empleados activos</h3>
    <ul>
      <Empleado nombre="Marcos" cargo="Ventas" activo={true} />
      <Empleado nombre="Noelia" cargo="Caja" activo={false} />
      <Empleado nombre="Orlando" cargo="Ventas" activo={true} />
      <Empleado nombre="Paula" cargo="Caja" activo={true} />
    </ul>
    </section>
  )
}
```

Listado de empleados activos

- Marcos - Ventas - Activo
- Noelia - Caja - Inactivo
- Orlando - Ventas - Activo
- Paula - Caja - Activo

Operador condicional ternario

Se puede mejorar utilizando un operador ternario condicional

```
function Empleado(empleado) {
  let estado=empleado.activo?"Inactivo":"Activo";
  return (<li>{empleado.nombre} - {empleado.cargo} - {estado}</li> )
}
```

En particular cuando no se quiere devolver nada se puede devolver null. Aunque nunca es buena práctica devolver algo que no se puede mostrar.

```
function EmpleadoActivo(empleado) {
  if (empleado.activo)
  {  return (<li>{empleado.nombre} - {empleado.cargo} </li> )  }
  else
  { return null  }
}

function Listado() {
  return (
    <section>
    <h3>Listado de empleados activos</h3>
    <ul>
      <EmpleadoActivo nombre="Marcos" cargo="Ventas" activo={true} />
      <EmpleadoActivo nombre="Noelia" cargo="Caja" activo={false} />
      <EmpleadoActivo nombre="Orlando" cargo="Ventas" activo={true} />
      <EmpleadoActivo nombre="Paula" cargo="Caja" activo={true} />
    </ul>
    </section>
  )
}
```

Listado de empleados activos

- Marcos - Ventas
- Orlando - Ventas
- Paula - Caja

No es buena práctica devolver *null* en un componente, lo más frecuente es incluir o excluir condicionalmente el componente en el JSX del componente padre.

Operador &&

En JSX, lo siguiente {condición && <Elemento />} funciona de la siguiente manera: si condición es verdadera renderiza <Elemento />, sino no renderiza nada ".

```
function EmpleadoActivo(empleado) {
    return (<li>{empleado.nombre} - {empleado.cargo} - {empleado.activo && "Activo"}</li>)
}
```

Listado de empleados activos

- Marcos - Ventas - Activo
- Noelia - Caja -
- Orlando - Ventas - Activo
- Paula - Caja - Activo

Método map()

ReactJS se prefiere el método map por sobre el foreach, porque el método map crea una nueva matriz con los resultados, y el foreach realiza una operación para cada elemento, pero no devuelve nada.

En ReactJS, a menudo transformamos matrices de datos en matrices de elementos, por lo que necesitamos un método que devuelva una nueva matriz.

Ejemplo: Muestra una tabla dado los datos de un arreglo de objetos.

```
let empleados = [{ nombre: 'Carla', cargo: 'Ventas', activo: true },
{ nombre: 'Fabio', cargo: 'Caja', activo: false },
{ nombre: 'Ileana', cargo: 'Ventas', activo: true },
{ nombre: 'Lucas', cargo: 'Ventas', activo: false },
{ nombre: 'Rocio', cargo: 'Caja', activo: true }
];

function FilasEmpleados() {
  return (
    empleados.map(function (empleado) {
      return <tr><td>{empleado.nombre}</td><td>{empleado.cargo}</td></tr>;
    }))
}
function TablaEmpleados() {
  return (
    <section>
      <h3>Listado de empleados</h3>
      <table>
        <thead>
          <tr><th>Empleado</th><th>Cargo</th></tr>
        </thead>
        <tbody>
            <FilasEmpleados />
        </tbody>
      </table>
    </section>
  )
}
```

Listado de empleados

Empleado	Cargo
Carla	Ventas
Fabio	Caja
Ileana	Ventas
Lucas	Ventas
Rocio	Caja

Key

En la ventana aparecerá una advertencia que una key debería ser proporcionada para ítems de lista. Una “key” es un atributo especial string que debes incluir al crear listas de elementos.

```
▶Warning: Each child in a list should have a unique "key" prop.

Check the render method of `Listado`. See https://reactjs.org/link/warning-keys for more inf
    at li
    at Listado
    at div
```

Las keys ayudan a React a identificar que ítems han cambiado, son agregados, o son eliminados. Deben ser dadas a los elementos dentro del array para darle a los elementos una identidad estable.

Las keys pueden ser tomadas de un dato clave o id que son únicas o se pueden generar localmente.

Deben ser únicas entre elementos hermanos y no deben cambiar. Nos permiten identificar de forma única un elemento entre sus hermanos. Permite a React identificar el elemento durante toda su vida útil.

Por ello podemos tener lo siguiente:

```
let empleados = [{ id: 1, nombre: 'Carla', cargo: 'Ventas', activo: true },
{ id: 2, nombre: 'Fabio', cargo: 'Caja', activo: false },
{ id: 3, nombre: 'Ileana', cargo: 'Ventas', activo: true },
{ id: 4, nombre: 'Lucas', cargo: 'Ventas', activo: false },
{ id: 5, nombre: 'Rocio', cargo: 'Caja', activo: true }
];
```

Puede ser que lo agreguemos directo a cada

```
function FilasEmpleados() {
  return (
    empleados.map(function (empleado) {
      return <tr key={empleado.id}><td>{empleado.nombre}</td><td>{empleado.cargo}</td></tr>;
    }))
}
```

Tenga en cuenta que sus componentes no recibirán key. React solo lo usa como una pista. Si su componente necesita una identificación, debe pasarla como un accesorio separado: <FilaEmpleado key={id} ...} />.

```
function FilaEmpleado({nombre,cargo}) {
  return <tr><td>{nombre}</td><td>{cargo}</td></tr>
}

function FilasEmpleados() {
  return (
    empleados.map(function (empl) {
      return <FilaEmpleado key={empl.id} nombre={empl.nombre} cargo={empl.cargo}/>;
    }))
}
```

Método filter()

Dada un arreglo de elementos, los compara a cada uno con una condición y en caso de que sea verdadera para ese elemento lo incluye en una nueva lista que devuelve.

```
let eVentas = empleados.filter(empl => empl.cargo == "Ventas");
let eCajas = empleados.filter(empl => empl.cargo == "Caja");

function FilasEmpleados({ lista }) {
  return ( lista.map(function (empl) {
   return <FilaEmpleado key={empl.id} nombre={empl.nombre} cargo={empl.cargo} />;
  }))  }
```

```
function TablaEmpleados({ lista, titulo }) {
  return (
    <section>
      <h3>Listado de empleados {titulo}</h3>
      <table>
        <thead>
          <tr><th>Empleado</th><th>Cargo</th></tr>
        </thead>
        <tbody>
          <FilasEmpleados lista={lista} />
        </tbody>
      </table>
    </section>
  )
}

const lugar = createRoot(document.getElementById('contenedor'));
lugar.render(
  <div>
    <TablaEmpleados lista={eVentas} titulo="Ventas" />
    <hr />
    <TablaEmpleados lista={eCajas} titulo="Caja" />
  </div>
);
```

Listado de empleados Ventas

Empleado	Cargo
Carla	Ventas
Ileana	Ventas
Lucas	Ventas

Listado de empleados Caja

Empleado	Cargo
Fabio	Caja
Rocio	Caja